

Cheat Sheet: Graphical Processing Units (GPUs) – Part 8

CPU vs GPU fundamentals

- **CPUs and SIMD retrofitting:** Vector instructions (SSE, AVX, AVX2, AVX-512) were retrofitted into traditional x86 CPUs. Compilers must ensure data alignment and absence of loop-carried dependencies, so vectorization is often limited. Some CPUs even throttle when using wide SIMD instructions, limiting performance gains.
- **GPU origins:** Early GPUs were designed to render 3D scenes for games and workstations. Manufacturers (ATI, NVIDIA, SGI, Sony) built GPUs to process pixels in parallel, leading to hardware with many lightweight threads. The same operation applied to millions of pixels makes GPUs naturally parallel.
- **Throughput vs latency:** GPUs sacrifice per-thread latency to maximize throughput. CPUs optimize for low-latency serial tasks, whereas GPUs use thousands of SIMD lanes to run many threads concurrently.

Performance trends

- **Transistor counts:** GPU transistor counts have followed a Moore's-Law-like exponential trend for ~30 years. Delays in manufacturing shrinks (28 nm→14/16 nm) caused plateaus in 2013-2015.
- **Peak FLOPS:** Peak double-precision performance rose from O(100) GFLOPS in mid-2000s to O(10,000) GFLOPS by 2018. GPUs deliver orders of magnitude more FLOPS than CPUs.
- **Energy efficiency:** GFLOPS per watt have increased over time; GPUs offer a much higher GFLOPS/W than CPUs.
- **Thermal design power (TDP):** GPU TDPs were limited to ~250 W by PCIe until proprietary connections allowed higher TDPs (e.g., Volta V100). Modern server CPUs have also increased TDP to compete.
- **Memory bandwidth:** GPU memory throughput far exceeds CPU memory throughput, enabling high data-transfer rates. This is the main reason GPUs excel at memory-bound tasks.
- **Arithmetic intensity:** Arithmetic intensity = FLOPs/byte. Modern GPUs require ≈ 10 double-precision FLOPs per byte to reach peak throughput. Common operations (vector add, axpy, dot product) have intensities $\ll 10$, meaning they are memory-bound.

Example GPU: NVIDIA Volta V100

- **Core count:** 80 streaming multiprocessors (SMs).
- **Double-precision units:** Each core has 32 FP64 units (2,560 total). Each unit performs fused-multiply-add (FMADD) per clock, yielding ≈ 7.8 TFLOPS at 1.53 GHz.
- **Tensor cores:** Specialized units perform half-precision tensor operations for machine-learning workloads.
- **Memory hierarchy:** Each core has 256 KB registers and 128 KB unified L1/shared memory; the chip has a 6 MB L2 cache and high-bandwidth memory.

GPU memory system & arithmetic intensity

- **Hierarchy:** Registers (per thread) → shared memory/L1 (per block) → L2 cache (shared) → global device memory. Registers and shared memory provide fast access; global memory accesses incur hundreds of cycles.
- **Peak arithmetic intensity:** To utilize GPU compute units, operations must perform many FLOPs per byte transferred. Table 8.1 shows intensities for typical double-precision operations: addition ≈ 0.04 , axpy ≈ 0.08 , dot product ≈ 0.12 , self dot ≈ 0.25 , matrix-vector ≈ 0.25 . Since these intensities are far below the ~ 10 required, these kernels are memory-bound.

Host-device model & CUDA programming basics

- **Heterogeneous system:** CPU (HOST) and GPU (DEVICE) have separate memory spaces. Data must be explicitly copied between them via PCIe or NVLink.
- **Programming steps:** Typical CUDA workflow: ① allocate arrays on HOST, ② allocate arrays on DEVICE using `cudaMalloc`, ③ copy data Host→Device with `cudaMemcpy`, ④ launch kernel (`kernel<<<G,B>>>`), ⑤ copy results back, ⑥ free memory.
- **Thread hierarchy:** Kernels execute on grids of thread-blocks. Each block contains threads. A thread can compute a linear index `n = threadIdx.x + blockIdx.x * blockDim.x`.
- **Unified Virtual Memory (UVM):** `cudaMallocManaged` allocates memory accessible on both host and device. Data migration happens automatically, but programmers must synchronize with `cudaDeviceSynchronize`.
- **Timing:** CUDA event timers (`cudaEventRecord`, `cudaEventElapsedTime`) allow accurate measurement of kernel execution time.

Kernel examples

- **fillKernel:** Each thread writes a value to one array element. Key intrinsics: `threadIdx.x`, `blockIdx.x`, `blockDim.x`.
- **Vector addition and multiplication:** Each thread loads elements from arrays and writes results; the mapping `n = t + b*B` determines which element a thread processes.
- **Reduction kernels:** Sequential kernel (V1) sums elements in one thread (slow). V2 uses a local accumulator to reduce memory traffic. V3 uses N threads but suffers race conditions. V4 uses `atomicAdd` to avoid races but incurs serialization. V5 and later variants use shared memory and tree reductions to improve performance (not fully detailed here).

Ongoing developments & future trends

- **Memory bandwidth vs compute:** Floating-point throughput is increasing faster than memory bandwidth; thus many GPU cores sit idle waiting for data.
- **Specialized cores:** To mitigate memory-bandwidth limits and broaden applications, GPUs now include tensor cores (for AI) and ray-tracing cores. These perform many operations per data element, improving arithmetic intensity.
- **Manufacturing challenges:** High-end GPUs require advanced fabrication (7 nm). Limited foundries (e.g., TSMC) and competition with smartphone chips cause supply constraints. Chiplet-based architectures may enable future scaling.

Structured Summary

8.1 Why efficient threading is tricky on CPUs

Most previous courses showed programming CPUs in serial C, with multithreading (OpenMP) and distributed computing (MPI). Achieving high parallel performance on CPUs is difficult because SIMD instructions (MMX, SSE, AVX, AVX-512) are retrofits to the x86 architecture. Compilers must ensure data alignment, full vector width, and absence of loop-carried dependencies before generating vector code. Updates to SIMD instruction sets require new code versions, and some CPUs throttle clock frequency when executing wide AVX-512 instructions. Thus, CPUs provide modest speed-ups for data-parallel tasks.

8.2 GPUs as an alternative to CPUs

The rise of home computers and gaming in the 1980s–90s drove development of GPUs specialized for rendering polygonal 3D scenes. GPUs used hierarchical parallelism from the start, enabling the same operation to run on many pixels concurrently. They became ubiquitous in desktops, laptops and phones as display resolutions increased. Their specialized role allowed simpler cores with many threads. This design produced exponential growth in transistor counts and peak FLOPS, although manufacturing plateaus occurred during process transitions.

8.3 Example GPU: the NVIDIA Volta V100

The Volta V100 serves as a concrete example of modern GPU architecture. It has 80 compute cores (streaming multiprocessors), each with 32 double-precision units and support for fused-multiply-add instructions. With a clock up to 1.53 GHz, the V100 can reach about 7.8 TFLOPS (double precision). Tensor cores provide high-throughput half-precision operations for machine learning. However, the massive number of FPUs often sits idle because the memory subsystem cannot supply data fast enough. Each core contains 256 KB registers and 128 KB unified shared/L1 memory; the chip has a 6 MB L2 cache and uses high-bandwidth memory.

8.4 GPU memory system & arithmetic intensity

GPU compute power exceeds its memory bandwidth. Figure 8.8 shows that to achieve peak performance, GPUs require about 10 double-precision FLOPs per byte of off-chip memory access. Typical operations like vector addition, axpy and dot product have arithmetic intensities far below this value, meaning they are memory-bound and utilize only a fraction of the FPUs. A GPU's memory hierarchy consists of registers, shared (on-core) memory/L1 cache, an L2 cache shared across cores, and global device memory. Accessing global memory takes hundreds of cycles; programmers must exploit shared memory and caches to hide latency.

8.5 Ongoing developments in GPU architectures

Successive GPU generations show dramatic improvements in floating-point throughput. Table 8.2 compares the Tesla c780 (2007), V100 (2017) and Hopper H200 (2024), revealing ~40× increase in FP32 throughput but only ~11× increase in memory bandwidth. This widening gap implies many cores are idle waiting for data.

Manufacturers address this by adding specialized units such as tensor cores (for AI) and ray-tracing cores (for graphics), each performing many operations per memory read. Fabrication challenges at 7 nm and supply constraints limit further scaling; chiplet designs and stacked memory may help in the future.

8.6 GPU programming models

GPUs transitioned from graphics accelerators to general-purpose compute devices when NVIDIA introduced CUDA (Compute Unified Device Architecture) in 2007. Programmers porting applications to CUDA often observed significant speed-ups due to both higher peak performance and the ease of writing parallel code. CUDA makes SIMD vectorization explicit; writing kernels forces developers to reason about thread and block indices. Programmers must allocate memory on the device, copy data, launch kernels and copy results back. Section 8.9 presents a “Hello World” example illustrating host code (`cudaMalloc`, `cudaMemcpy`, `cudaFree`) and device kernel (`fillKernel`). Unified Virtual Memory (UVM) introduced by `cudaMallocManaged` simplifies memory management by automatically migrating data, but may incur overhead. CUDA event timers allow accurate measurement of kernel execution time.

8.9–8.14 Vector operations and API summary

CUDA kernels map thread-block/thread indices to linear array indices. Examples include vector addition and element-wise multiplication, where each thread processes one element. Table 8.3 summarizes common CUDA API calls: `cudaMalloc`, `cudaMallocManaged`, `cudaMemcpy`, `cudaFree`, kernel launch syntax, and `cudaDeviceSynchronize`. These primitives form the backbone of many CUDA programs.

8.15–8.16 Memory hierarchy and reduction kernels

Table 8.4 summarizes the GPU memory hierarchy: host memory (~1 TB), device global memory (16–32 GB), L2 cache (6 MB), unified L1/shared memory (128 KB per core), and registers (4×64 KB per core). Access latencies increase dramatically from registers to global memory; caching reduces latency but programmers have limited control. Reduction kernels illustrate different approaches to summing array elements: a naive single-thread kernel (V1) uses global memory and is extremely slow; V2 uses a local accumulator; V3 introduces multiple threads but suffers race conditions; V4 uses `atomicAdd` to avoid races at the cost of serialization. Later kernels (not detailed here) leverage shared memory, partial reductions per block and tree-based reductions to improve bandwidth utilization.

Multiple-Choice Questions (1–30)

Each question has four options (A–D) with exactly one correct answer.

1. **Why is vectorization difficult on traditional CPUs?** A. CPUs lack SIMD instructions entirely.
B. Compilers must ensure data alignment and absence of loop dependencies before using SIMD instructions.
C. CPUs cannot execute floating-point operations.
D. SIMD instructions only exist on GPUs.

2. **Early GPUs were primarily designed to:** A. Run operating systems.
B. Accelerate 3D graphics rendering by processing pixels in parallel.
C. Replace CPUs in desktop computers.
D. Manage hard-disk I/O.
3. **Which statement best describes the throughput-latency trade-off in GPUs?** A. GPUs maximize single-thread latency and reduce throughput.
B. GPUs prioritize throughput over per-thread latency by running many lightweight threads.
C. GPUs and CPUs have identical latency characteristics.
D. GPUs only perform one task at a time.
4. **According to the lecture, GPU transistor counts:** A. Have stagnated since the 1990s.
B. Follow a Moore's-Law-like exponential trend, with plateaus during manufacturing transitions.
C. Decrease with each generation.
D. Are irrelevant to performance.
5. **Peak double-precision FLOPS for GPUs grew from O(100 GFLOPS) in the mid-2000s to approximately:** A. 10 GFLOPS by 2018.
B. 1,000 GFLOPS by 2018.
C. 10,000 GFLOPS by 2018.
D. 1 GFLOP by 2018.
6. **GFLOPS per watt for GPUs relative to CPUs:** A. Is much lower for GPUs.
B. Is roughly the same for both.
C. Shows GPUs offer higher energy efficiency than CPUs.
D. Cannot be compared.
7. **What limits GPU power consumption when connected via PCIe?** A. PCIe devices were limited to ≈ 250 W, leading to TDP plateaus until proprietary connections removed the limit.
B. GPU power is limited by CPU voltage regulators.
C. GPUs cannot exceed 50 W by design.
D. Memory bandwidth limitations restrict power.
8. **The term *arithmetic intensity* refers to:** A. The ratio of bytes moved per second.
B. The number of floating-point operations per byte of data moved.
C. GPU clock speed.
D. The number of threads per block.
9. **Modern GPUs require an arithmetic intensity of roughly 10 FLOPs per byte to reach peak performance. Which operation has the highest intensity among these?** A. Vector addition ($z_i = x_i + y_i$).
B. Axdy ($y_i = \alpha x_i + y_i$).
C. Dot product ($a = \sum x_i y_i$).
D. Matrix-vector multiplication ($y_i = \alpha \sum A_{ij} x_j + \beta y_i$).
10. **In the NVIDIA Volta V100, the total number of double-precision floating-point units (FPUs) is:** A. 80.

- B. 32.
 - C. 2,560.
 - D. 256.
11. **A *tensor core* in GPUs is used primarily for:** A. Integer arithmetic.
B. Half-precision matrix operations common in machine learning.
C. Controlling memory transfers.
D. Handling network packets.
 12. **Which memory hierarchy level is fastest and closest to a GPU thread?** A. Global device memory.
B. Registers.
C. L2 cache.
D. Unified shared/L1 memory.
 13. **What happens if a GPU kernel reads data not present in L1 or L2 cache?** A. It retrieves the data from registers instantly.
B. The thread stalls while the data is fetched from global memory, incurring hundreds of cycles of latency.
C. The data is discarded.
D. The kernel restarts.
 14. **In CUDA, `cudaMalloc` is used to:** A. Allocate host memory.
B. Allocate device memory on the GPU.
C. Launch a kernel.
D. Copy data from device to host.
 15. **A typical CUDA program sequence is:** A. Launch kernel → allocate host memory → free memory.
B. Copy host to device → allocate device memory → free memory.
C. Allocate host and device memory → copy data to device → launch kernel → copy results back → free memory.
D. Free memory → allocate device memory → copy results back.
 16. **The linear index calculation `n = threadIdx.x + blockIdx.x * blockDim.x` maps each thread to:** A. A block index only.
B. An element of an array or workload.
C. The entire grid.
D. The GPU host memory.
 17. **Unified Virtual Memory (UVM) simplifies programming by:** A. Eliminating the need for host memory.
B. Automatically migrating data between host and device when pointers are used.
C. Disabling synchronization between CPU and GPU.
D. Reducing memory capacity on GPUs.
 18. **Why must `cudaDeviceSynchronize()` be called when using UVM in the CUDA Hello World example?** A. To allocate more GPU threads.

- B. To ensure the kernel has finished before the host reads managed memory.
 - C. To deallocate memory.
 - D. To convert pointers from host to device.
19. **CUDA event timers (`cudaEventRecord`, `cudaEventElapsedTime`) are useful because:** A. GPU clocks cannot be read directly.
- B. Kernels execute asynchronously, so timers accurately measure elapsed time between event insertion and completion.
 - C. They launch kernels automatically.
 - D. They allocate device memory.
20. **The `fillKernel` example demonstrates:** A. Serial execution of array filling.
- B. Mapping thread and block indices to compute a unique array element for each thread.
 - C. Printing results from the device.
 - D. Allocating unified memory.
21. **In reduction kernel V2, performance is improved over V1 by:** A. Using more threads.
- B. Using a local accumulator variable instead of a global memory location.
 - C. Introducing race conditions.
 - D. Disabling caches.
22. **Reduction kernel V3 is considered bad code because:** A. It uses no threads.
- B. It causes read-write race conditions when multiple threads update a single accumulator.
 - C. It allocates too much memory.
 - D. It performs no addition.
23. **Which CUDA function atomically adds a value to a memory location to avoid race conditions?**
- A. `cudaMalloc`
 - B. `atomicAdd`
 - C. `cudaEventRecord`
 - D. `cudaDeviceSynchronize`
24. **Shared memory on the GPU:** A. Is slower than global memory.
- B. Is private to one thread only.
 - C. Is a fast scratchpad accessible to all threads in a block and is explicitly managed by the programmer.
 - D. Does not exist on modern GPUs.
25. **Why are tensor cores and ray-tracing cores being added to GPUs?** A. To reduce manufacturing cost.
- B. To increase arithmetic intensity and target specialized applications like AI and graphics.
 - C. To replace double-precision FPUs.
 - D. To eliminate the need for memory.
26. **Which of the following correctly pairs host and device roles?** A. HOST = GPU processor; DEVICE = CPU and main memory.

- B. HOST = CPU with host memory; DEVICE = GPU with its own memory.
C. HOST and DEVICE share the same physical memory.
D. DEVICE can directly access host memory without copying.
27. **When using CUDA, why should data movement between host and device be minimized?** A. Because copy operations are faster than arithmetic operations.
B. Because host-device transfers are usually much slower than device-device memory access and can bottleneck performance.
C. Because devices have infinite memory.
D. Because CUDA automatically avoids transfers.
28. **What is a key reason GPUs are taking over data-center workloads such as machine learning and simulations?** A. GPUs have lower transistor counts.
B. GPUs have higher memory throughput and parallelism, enabling faster processing of memory-bound tasks.
C. CPUs are no longer manufactured.
D. GPUs consume more power.
29. **Which of the following statements about unified memory is TRUE?** A. It always outperforms manual memory transfers.
B. It may introduce performance overhead due to just-in-time data migration.
C. It requires kernel synchronization to be disabled.
D. It does not support multi-GPU systems.
30. **What challenge does the lecture mention regarding future GPU development?** A. There is too much competition among foundries.
B. Memory bandwidth is increasing faster than compute performance.
C. Manufacturing costs and the supply of advanced process nodes (e.g., 7 nm) limit GPU scaling.
D. GPUs will no longer support double-precision arithmetic.
-

Answer Key & Explanations

1. **B** – Compilers must verify data alignment and loop dependencies before emitting SIMD instructions.
2. **B** – Early GPUs were specialized to render graphics by performing the same operation on many pixels.
3. **B** – GPUs trade high per-thread latency for massive throughput by running many lightweight threads in parallel.
4. **B** – GPU transistor counts grow exponentially but plateau during process transitions.
5. **C** – Peak double-precision performance reached O(10,000 GFLOPS) by 2018.
6. **C** – GPUs deliver more GFLOPS per watt than CPUs.
7. **A** – PCIe limits devices to ≈ 250 W; proprietary interfaces (e.g., NVLink) allowed higher TDPs.
8. **B** – Arithmetic intensity = floating-point operations divided by bytes of data moved.
9. **D** – Among the operations listed, matrix-vector multiplication (~ 0.25 FLOPs/byte) has the highest intensity.
10. **C** – The V100 has 2,560 FP64 units (80 cores $\times$ 32).

11. **B** – Tensor cores accelerate half-precision matrix operations common in machine learning.
 12. **B** – Registers are the fastest memory and reside within each thread.
 13. **B** – A cache miss triggers a global memory access, which incurs hundreds of cycles of latency.
 14. **B** – `cudaMalloc` allocates memory on the device (GPU).
 15. **C** – Allocate host and device memory, copy data to device, launch kernel, copy results back, then free memory.
 16. **B** – The index formula maps each thread to a unique array element to work on.
 17. **B** – UVM automatically migrates data between host and device when a pointer is accessed.
 18. **B** – Synchronization ensures the kernel finishes and data is migrated before the host reads the managed array.
 19. **B** – Events measure elapsed time between insertion and completion of asynchronous kernel execution.
 20. **B** – `fillKernel` maps thread and block indices to an array index and assigns work accordingly.
 21. **B** – V2 uses a local accumulator to avoid repeated updates to global memory.
 22. **B** – V3 allows many threads to update a single variable without synchronization, causing race conditions.
 23. **B** – `atomicAdd` performs an uninterruptible addition to avoid race conditions.
 24. **C** – Shared memory is a fast, programmer-managed scratchpad accessible to all threads in a block.
 25. **B** – Specialized tensor and ray-tracing cores increase arithmetic intensity and target new workloads (AI, graphics).
 26. **B** – The host (CPU) with host memory communicates with the device (GPU), which has its own memory.
 27. **B** – Host-device transfers are an order of magnitude slower than device memory access, so minimizing data movement is vital.
 28. **B** – GPUs offer higher memory bandwidth and parallelism, making them attractive for data-center workloads.
 29. **B** – Unified memory simplifies programming but may incur overhead due to automatic data migration.
 30. **C** – Limited foundry capacity and manufacturing costs at advanced process nodes (e.g., 7 nm) pose challenges for future GPUs.
-